

Manual Técnico y Arquitectónico Completo: InciTrack & GranCRM

1. Introducción al Ecosistema

InciTrack es un módulo de gestión de incidencias (Tickets) diseñado para operar dentro del ecosistema **GranCRM**. En lugar de ser una aplicación monolítica tradicional, InciTrack está dividido en dos partes principales:

1. **Un Frontend Desacoplado (React/Vite):** Que funciona como un "Micro-Frontend".
2. **Un Backend Dedicado (Django):** Que gestiona la lógica de negocio y se conecta a su propia base de datos, independiente de la del Orquestador.

Este enfoque permite que el equipo de InciTrack pueda desplegar actualizaciones, reiniciar sus servidores o modificar su base de datos sin afectar a otras aplicaciones (como CallReviews o el propio Orquestador).

2. Arquitectura de Micro-Frontends (Module Federation)

El frontend de InciTrack utiliza **Webpack Module Federation** (mediante Vite). Esto resuelve el problema de cómo cargar múltiples aplicaciones de React dentro de una sola ventana del navegador.

¿Cómo funciona la inyección?

1. El usuario inicia sesión en el **Orquestador**.
2. El Orquestador lee el archivo de manifiesto (**dios.json**) de InciTrack para saber dónde está su código compilado.
3. El navegador descarga el archivo **remoteEntry.js** de InciTrack.
4. El Orquestador inyecta los componentes de InciTrack en la pantalla, pasándole **Propiedades Clave (Props)**:
 - **session**: Los datos del usuario logueado.
 - **apiBase**: La URL donde InciTrack debe hacer sus peticiones backend (**/incitrack/**).
 - **basename**: La ruta del navegador (**/incitrack**).
 - **bus**: Un canal de eventos para comunicarse con el Orquestador (ej. avisar si la sesión expiró).

3. Conexión a Base de Datos (SQL Server)

InciTrack se conecta a una base de datos **Microsoft SQL Server 2019**, específicamente a la instancia **QAIntouch** alojada en la IP **172.20.21.50**.

Detalles de la Conexión (Backend)

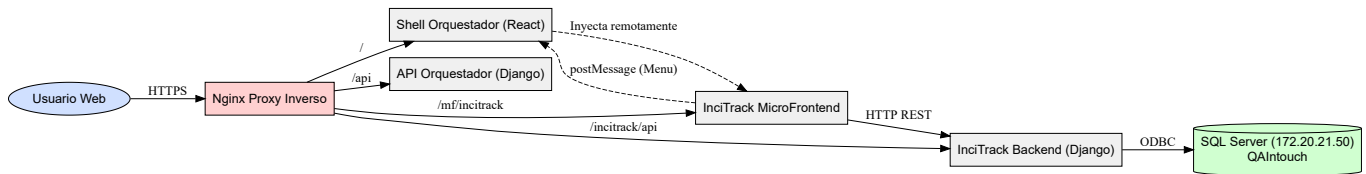
- **Motor:** **mssql** (vía ODBC Driver 18)
- **Base de Datos:** **QAIntouch**
- **Usuario:** **incitrack_login**
- **Puerto:** **1433**

Sincronización de Usuarios

Para mantener la integridad de los datos (saber quién creó un ticket), InciTrack necesita tener a los usuarios en su propia base de datos. En lugar de conectar InciTrack a la BD del Orquestador (lo cual sería un antipatrón), **InciTrack crea una copia del usuario al vuelo**. Cuando un usuario entra a InciTrack por primera vez, el archivo **grancrm_session.py** lee su token, extrae su email y nombre, y lo guarda en la tabla de usuarios de **QAIntouch**.

4. Diagrama de Flujo y Arquitectura

Para que el mapa visual se imprima perfectamente en el PDF, hemos renderizado el flujo en la siguiente imagen de alta resolución:



Flujo de Peticiones (Paso a Paso)

1. **[Navegador]** -> El usuario entra a `https://qadash.../incitrack/`
2. **[Nginx]** -> El proxy Nginx intercepta la petición. Como la URL es `/mf/incitrack/`, Nginx va a la carpeta de Vite y devuelve los archivos `.js` y `.css` de React.
3. **[Navegador]** -> React carga la interfaz y hace un `fetch()` a `/incitrack/api/v1/tickets/`.
4. **[Nginx]** -> Nginx ve `/incitrack/api/` y redirige la petición al contenedor Docker del backend de InciTrack (puerto 8000).
5. **[Backend Django]** -> El middleware `grancrm_session.py` recibe la petición. Extrae la cookie `grancrm_session`, la descrypta y verifica que el usuario es válido.
6. **[SQL Server]** -> Django se conecta a `172.20.21.50`, busca los tickets, y devuelve los datos en JSON.

5. Comparativa de Entornos: DEV (.248) vs QA (.249)

El entorno de **QA (.249)** fue actualizado recientemente para independizar su menú lateral. El entorno de **DEV (.248)** aún mantiene el código antiguo acoplado al Orquestador.

Aspecto	QA (.249) - Independizado	DEV (.248) - Acoplado
Control del Menú (Sidebar)	Frontend de InciTrack. El archivo <code>App.tsx</code> calcula el menú usando <code>postMessage</code> .	BD del Orquestador. Depende de la tabla Aplicacion y lo que dictó <code>dios.json</code> .
Manejo de Roles	Transparente. Traduce internamente <code>admin_cuenta</code> a <code>admin</code> antes de validar accesos.	Problemático. Si el Orquestador pasa un rol nuevo, InciTrack le niega el acceso.
Dependencia del Orquestador	Mínima. Solo usa el Token JWT y el CSS global.	Alta. Requiere sincronización manual de la base de datos para mostrar el menú.

6. Diccionario de Archivos Clave y sus Funciones

Frontend (Carpeta: `frontend/src/`)

1. **App.tsx**: El núcleo de React. Configura las rutas (`react-router`), normaliza los roles de GranCRM e inyecta dinámicamente el menú lateral enviando el evento `grancrm:nav`.
2. **context.tsx**: Provee el "Contexto" a toda la aplicación. Aquí vive la función `normalizeRole` que unifica los roles viejos y nuevos.
3. **api.ts**: El interceptor de red. Todas las peticiones al backend pasan por aquí. Adjunta el `X-CSRFToken` y maneja los errores 401 (Sesión Expirada).
4. **main.tsx**: Solo se usa en desarrollo local. Simula un Orquestador falso (DevShell) para que los desarrolladores puedan trabajar sin levantar todo GranCRM.
5. **RoleGuard.tsx**: Componente protector. Envuelve partes de la interfaz (como los botones de Admin) y los oculta si el usuario no tiene el rol necesario.

Backend (Carpeta: `incitrack/`)

1. **incitrack/settings.py**: Archivo maestro de Django. Define la conexión ODBC a SQL Server, contraseñas, URLs del Orquestador y claves JWT.
2. **tickets/grancrm_session.py**: El guardián de seguridad. Middleware que lee la cookie del usuario, verifica que la firma JWT sea correcta y sincroniza el usuario en la BD de SQL Server.
3. **tickets/api.py**: Controladores de la API (Vistas). Aquí se construyen los JSON que el frontend consume.
4. **tickets/models.py**: Definición de las tablas de SQL Server (Tickets, SLA, Notificaciones) mapeadas a código Python.
5. **dios.json**: Manifiesto técnico. Antes controlaba el menú, ahora solo sirve para decirle al Orquestador dónde está la IP de InciTrack.

7. Comandos Útiles para el Día a Día

Comandos de Docker (Infraestructura)

```
# Ver si el contenedor de InciTrack está corriendo y sano
sudo docker compose ps | grep incitrack

# Reiniciar InciTrack para aplicar cambios en el Backend (Python)
sudo docker compose restart incitrack-modulo

# Ver los logs en tiempo real (útil para ver errores de SQL Server)
sudo docker compose logs -f incitrack-modulo
```

Comandos de Frontend (React)

```
# Entrar a la carpeta del frontend
cd /home/admincrm/grancrm/incitrack/frontend

# Instalar dependencias si se agregó un nuevo paquete
npm install

# Compilar el código a producción (Obligatorio tras hacer cambios en .tsx)
npm run build
```

Comandos de Backend (Django)

```
# Entrar dentro del contenedor de InciTrack
sudo docker compose exec incitrack-modulo bash

# Crear nuevas migraciones tras modificar models.py
python manage.py makemigrations

# Aplicar las migraciones a SQL Server (172.20.21.50)
python manage.py migrate

# Limpiar sesiones expiradas en la BD
python manage.py clearsessions
```

8. Consideraciones para Producción

1. **Variables de Entorno:** Nunca hardcodear contraseñas de SQL Server en `settings.py`. Siempre usar el `.env`.
2. **Caché del Navegador:** Vite hace *cache busting* automático, pero el archivo `remoteEntry.js` a veces se queda pegado en la memoria de Chrome. Tras cada despliegue, recuerda presionar **Vaciar caché y**

volver a cargar de manera rígida.

3. **Mantenimiento del JWT:** Si el Orquestador rota o cambia la variable `GRANCRM_JWT_SECRET`, InciTrack dejará de funcionar (Tirará Error 401 en todo) hasta que actualices la misma variable en su `.env`.